

To integrate `NextResponse.next()` with custom request headers and `next-intl`'s `createMiddleware`, you can wrap the internationalization logic in a custom middleware function. This allows you to modify the request headers before passing them to the `i18n` handler or modify the resulting response. [🔗](#)

Implementation Guide

- **Modify Headers Before `i18n`:** Create a new `Headers` object and pass it to the target page via `NextResponse.next`. To keep these headers available while using `next-intl`, you must ensure the response from `createMiddleware` is returned correctly. [Functions: NextResponse - Next.js](#)
- **Compose the Middleware:** Use a wrapper function to handle both custom header logic and internationalized routing. [Proxy / middleware – next-intl](#) [🔗](#)

Example Code

typescript

```
import createMiddleware from 'next-intl/middleware';
import { NextRequest, NextResponse } from 'next/server';
import { routing } from './i18n/routing';

const handleI18nRouting = createMiddleware(routing);

export default function middleware(request: NextRequest) {
  // 1. Prepare custom request headers
  const requestHeaders = new Headers(request.headers);
  requestHeaders.set('x-custom-header', 'my-value');

  // 2. Run i18n middleware
  // Note: next-intl usually handles redirects/rewrites based on the request
  const response = handleI18nRouting(request);

  // 3. Apply custom request headers to the final response
  // Use NextResponse.next to ensure headers are forwarded upstream
  const finalResponse = NextResponse.next({
    request: {
      headers: requestHeaders,
    },
  });

  // 4. Merge i18n-specific headers (like x-middleware-rewrite)
  // if you need to preserve the locale routing determined by next-intl
  response.headers.forEach((value, key) => {
```

```
    finalResponse.headers.set(key, value);
  });

  return finalResponse;
}

export const config = {
  // Match only internationalized pathnames
  matcher: ['/', '/(de|en)/:path*']
};
```

Key Considerations

- **Header Forwarding:** Using `NextResponse.next({ request: { headers } })` forwards headers upstream to your pages and Server Actions but does **not** expose them to the client browser. Functions: `NextResponse` - `Next.js`
- **Order of Execution:** If you need the `next-intl` middleware to see your custom headers, modify `request.headers` *before* calling `handleI18nRouting(request)`. Proxy / middleware – `next-intl`
- **Response Headers vs Request Headers:** Standard response headers (e.g., `Set-Cookie`) should be added directly to the `response` object returned by `handleI18nRouting`. [Middleware - Next.js](#)